

Sri Lanka Institute of Information Technology

Faculty of Computing

SE2012 - Object Oriented Analysis and Design

Prof. Jagath Samarabandu
Mr. Jeewaka Perera

Year 02 and Semester 01

Lecture 02

Introduction to Requirement Engineering

Textbook: *Requirements Engineering for Software and Systems, 4th Edition* (oreilly.com).

Chapter 1: Introduction to Requirements Engineering

Objectives

- Understand the basics of Requirements Engineering.
- Interpret and create Use Case Diagrams.
- Understand Use Case Scenarios and User Stories.
- Application of concepts to real world Scenarios.

Requirements

- To provide a solution to any given problem, we need to understand requirements.
- Requirements specify what is expected from the solution.
- Requirements can range from
 - high-level,
 - abstract statements,
 - back-of-the-napkin sketches
 - formal (mathematically rigorous) specifications.
- These varying representation forms occur because stakeholders have needs at different levels and hence depend on different abstraction representations.
- Stakeholders also have varying abilities to make and read these representations (e.g., a business customer vs. a design engineer), leading to diverse quality in the requirements.

Requirements vs. Features vs. Goals

- People often confuse requirements, features, and goals.
- Goals are high-level objectives of a business, organization, or system.
- Requirement specifies how a goal should be accomplished by a proposed system
- A features is a set of logically related requirements that allows a user to satisfy the goal

Example

- Consider a “*Smart home system*,”
 - A Goal is to automate that which does not need human interaction, to free the occupants to enjoy themselves with other activities.
 - This goal can be satisfied by implementing some features.
 - One of these features could be by providing automated water purification to the system.
 - Then, there will be a set of specific requirements to describe the water purification mechanism.
- The requirements tend to be more granular than a feature.
- Written with the implementation in mind.

Requirement Classification

- Requirements can be classified based on different criteria.
- Two most common Criteria are
 - Requirements Level Classification
 - User requirements
 - System requirements
 - Design specifications
 - Requirements Specifications Types
 - Functional requirements (FRs)
 - Nonfunctional requirements (NFRs)
 - Domain requirements

Requirements by Level

■ User requirements

- Abstract statements written in natural language with accompanying informal diagrams.
- They specify what services (user functionality) the system is expected to provide and any constraints.
- Collected user requirements often appear as a “concept of operations” (Conops) document.
- **user stories** can play the role of user requirements.

System requirements

- Detailed descriptions of the services and constraints.
- System requirements are sometimes referred to as functional specifications (or technical annexes).
- These requirements are derived from analysis of the user requirements, are structured and precise.
- The requirements are collected in a systems requirements specification (SRS) document.
- **Use cases** can play the role of system requirement in many situations.

Design Specifications

- Emerge from the analysis and design documentation used as the basis for implementation by developers.
- The system design specification is essentially derived directly from the analysis of the system requirements specification.

Example

- Consider a Point-of-Sale system
- A user requirement
 - The system shall accurately compute sale totals including discounts, taxes, refunds, and rebates; print an accurate receipt; and update inventory counts accordingly.
- Some related system requirements
 - Each sale shall be assigned a sales ID.
 - Each sale may have one or more sales items.
 - Each sale may have one or more rebates.
 - Each sale may have only one receipt printed.
- Software specifications
 - 1.2 The system shall assign a unique sales ID number to each sale transaction.
 - 1.2.1 Each sales ID may have zero or more sales items associated with it, but each sales item must be assigned to exactly one sales ID
 -

Functional Requirements

- Functional requirements (FRs) describe the services the system should provide and how the system will react to its inputs.
- FRs need to explicitly state certain behaviors that the system should not do.
- FRs can be high level and general (in which case they are user requirements in the sense that was explained previously) or they can be detailed, expressing inputs, outputs, exceptions, and so on (in which case they are the system requirements described before).

Non Functional Requirements

- Systems can be characterized by their non-functional behavior.
- In the software/system marketplace, in which functionally equivalent products compete for the same customer, non-functional requirements (NFRs) become more important in distinguishing between the competing products.
 - Security
 - Reliability
 - Maintainability
 - Performance
 - Usability
 - Testability
 - Interoperability
 - Constraints (e.g., design/implementation constraints, financial constraints, resources constraints)

Introduction to Requirements Engineering

- Requirements engineering is the branch of engineering concerned with the real-world goals for, functions, and constraints on systems.
- It involves
 - Defining,
 - Documenting
 - Maintaining requirements.

Requirement Engineering Process

- **Requirements elicitation/discovery**
- Requirements analysis and reconciliation
- Requirements representation/modeling
- Requirements verification and validation
- Requirements management

Requirements Elicitation/Discovery

- Requirements elicitation/discovery involves uncovering what the customer needs and wants.
- Some requirements will be obvious (e.g., the POS system will have to compute sales tax),
- Many requirements will need to be extricated from the customer through well-defined approaches.
- Involves discovering who the stakeholders are.
 - Are there any hidden stakeholders?
- Elicitation also involves determining the NFRs, which are often overlooked.

Obstacles in Requirements Elicitation

- New Project Domain:
 - when requirements engineer doesn't possess enough knowledge on the industry or the developed solution.
 - Solution : Engaging a domain expert can help alleviate this problem.
- Unclear Project Vision:
 - when stakeholders don't have a clear understanding of what functionality their system needs.
 - Solution : A clear mission statement or ConOps can be very helpful to avoid this problem.
- Limited Access to Documentation:
 - when the requirements engineer can't access documentation
 - when evaluating the current state of the project takes too much time.
 - Solution : Consistent and disciplined use of a good document/change management system is important to combat this problem.

Obstacles in Requirements Elicitation

- Focusing on the Solution Instead of Requirements:
 - when the customer focuses more on solutions or architectural tactics instead of requirements themselves.
 - Solution: Active expectation management is very important in this regard.
- Fixation of Specific Functionalities:
 - when stakeholders insist on designing certain features because they believe they will benefit their business.
 - Solution: Expectation management and refocus on the mission statement is the key to addressing this issue.
- Contradictory Requirements:
 - when a project includes a wide range of stakeholders, their requirements may contradict each other.

Gathering Requirements

- There may be various stakeholders for a particular system.
- These stakeholders will have different desires (Desiderata) or requirements from the system.
- Further, there could be various sub groups of stakeholders as well.

Steps in Gathering Requirements

- **Identify all customers and stakeholders.**
- **Partition customers and other stakeholders into classes according to interests, scope, authorization, or other discriminating factors (some classes may need multiple levels of partitioning).**
- Select a champion or representative group for each user class and stakeholder group.
- Select the appropriate technique(s) to solicit initial inputs from each class or stakeholder group.

Example

- In the Point of Sale (POS) system, who would be the different users/Stakeholders of this system
 - Cashiers
 - Managers
 - System maintenance personnel
 - Store customers
 - Inventory/warehouse personnel
 - Accountants (to enter tax information)
 - Sales department (to enter pricing and discounting information)

Activity 1

Think of a Airline Baggage Handling System. List the different Stakeholders of the system, including any subgroups.

Activity Discussion

- Travelers
 - Children
 - Senior citizens
 - Business people
 - Casual travelers
 - Military personnel
 - Civilians
 - Casual travelers
 - Frequent flyers
- System maintenance personnel
- Baggage handlers
- Airline schedulers/dispatchers
- Airport personnel
- Airport managers and policymakers

Steps in Gathering Requirements

- Identify all customers and stakeholders.
- Partition customers and other stakeholders into classes according to interests, scope, authorization, or other discriminating factors (some classes may need multiple levels of partitioning).
- **Select a champion or representative group for each user class and stakeholder group.**
- **Select the appropriate technique(s) to solicit initial inputs from each class or stakeholder group.**

Techniques for Gathering Requirements

- Brainstorming
- Card sorting
- Crowdsourcing
- Designer as apprentice
- Domain analysis
- Ethnographic observation
- Goal-based approaches
- Group work
- Interviews
- Introspection
- Joint application development (JAD)
- Laddering
- Protocol analysis
- Prototyping
- Quality function deployment (QFD)
- Questionnaires
- Repertory grids
- Reverse engineering
- **Scenarios**
- Task analysis
- **Use cases**
- **User stories**
- Viewpoints
- Workshops

Requirement Engineering Process

- Requirements elicitation/discovery
- **Requirements analysis and reconciliation**
- Requirements representation/modeling
- Requirements verification and validation
- Requirements management

Requirements Analysis and Reconciliation

- After requirements have been collected there will be several problems with requirements in their “raw” form, that is, after they have been collected from the customers.
 - They don’t always make sense.
 - They may contradict one another (and not always obviously so).
 - They may be inconsistent.
 - They may be incomplete.
 - They may be vague or just wrong.
 - They may interact and be dependent on each other.
- Many of the elicitation techniques that we will discuss subsequently are intended to avoid or alleviate these problems.
- Formal methods are also useful in this regard.

Requirement Engineering Process

- Requirements elicitation/discovery
- Requirements analysis and reconciliation
- **Requirements representation/modeling**
- Requirements verification and validation
- Requirements management

Requirements representation/modeling

- Requirements representation (or modeling) involves converting the processed requirements into some model (usually natural language, mathematics, and visualizations).
- Proper representations facilitate communication of requirements and conversion into a system architecture and design.
- Various techniques are used for requirements representation,
 - Informal (e.g., natural language, sketches, and diagrams)
 - Formal (mathematically sound representations)
 - Semiformal (convertible to a sound representation or can be made fully formal by the addition of a semantic framework).
- Usually, some combination of these is employed in requirements representation.

Requirement Engineering Process

- Requirements elicitation/discovery
- Requirements analysis and reconciliation
- Requirements representation/modeling
- **Requirements verification and validation**
- Requirements management

Requirements verification and validation

- Requirements validation is the process of determining if the specification is a correct representation of the customers' needs.
- Validation answers the question "Am I building the right product?"
- Requirements validation involves various semiformal and formal methods, text-based tools, visualizations, inspections and etc.

Requirement Engineering Process

- Requirements elicitation/discovery
- Requirements analysis and reconciliation
- Requirements representation/modeling
- Requirements verification and validation
- **Requirements management**

Requirements Management

- One of the most overlooked aspects of requirements engineering.
- It involves managing the realities of changing requirements over time.
- It also involves fostering traceability through appropriate aggregation and subordination of requirements and communicating changes in requirements to those who need to know.
- Managers also need to learn the skills to intelligently push back when scope creep ensues.
- Using tools to track changes and maintain traceability can significantly ease the burden of requirements management.

User Scenario

User Scenarios

- Scenarios are informal descriptions of the system in use that provide a high-level description of system operation, classes of users, and exceptional situations.
- Scenarios are quite useful when the domain is novel (consider a scenario for the space station, for example)
- Provides detailed descriptions of how users interact with a system to achieve specific goals.
- Helps in understanding user requirements and the context of system usage.

Example

A customer walks into the pet store and fills the cart with a variety of items. When checking out, the cashier asks if the customer has a loyalty card. If so, the cashier swipes the card, authenticating the customer. If not, then the cashier offers to complete one on the spot.

After the loyalty card activity, the cashier scans products using a bar code reader. As each item is scanned, the sale is computed, and the inventory is appropriately updated. Upon completion of product scanning a subtotal is computed. Then any coupons and discounts are entered. A new subtotal is computed, and applicable taxes are added. A receipt is printed, and the customer pays using cash, credit card, debit card, or check. All appropriate totals (sales, tax, discounts, rebates, etc.) are computed and recorded.

Activity 2

Form Small Groups and Try to come up with at least 2 Scenarios for Different Stakeholders you identified for the Baggage Handling System.

User Stories

User Stories

- User Stories are an extension of Scenarios narrated from the point of a user.
- Short, simple descriptions of a feature told from the perspective of the user.
- Helps in agile development to capture user needs quickly.
- User stories are generally well structured: Who, What, Why
 - As a [user], I want [goal] so that [benefit].
 - User: Who is using the feature.
 - Goal: What the user wants to achieve.
 - Benefit: Why the user wants to achieve this goal.

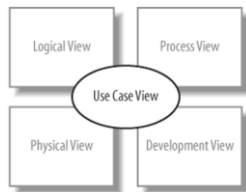
Example

- **As a:** Library member
- **I want:** To renew a borrowed book online
- **So that:** I don't have to visit the library in person

Use Cases

Use Cases

- Use cases are a way to capture and document the functional requirements of a system by describing the interactions between a user (or actor) and the system to achieve a goal.
- They help in understanding what the system should do from the user's perspective.
- A use case should provide some measurable result to the user or an external system.
- Use cases are at the heart of your model, since they affect and guide all of the other elements within your system's design.



Use Cases Continued

- Use cases are usually a good Starting Point for Object Oriented System Design Process.
- It describe a system's requirements strictly from the outside looking in.
- It specify the value that the system delivers to users.
- They should be the first serious output from your model after a project is started.
- You cannot begin to design a system if you don't know what it will be required to do.
- Use cases are a means to bring the gaps in the user's requirements to the forefront at the beginning of a project.
- Once a gap has been identified, go back to the system's stakeholders.
- When a requirement is presented as a use case, and the stakeholder might see that the requirement has little or no value to the system.
- Development will cost far less in both time and money

Use Cases – Advantages

- Assign priorities and risks associated with each Use Case.
 - Help manage a project's workload.
 - Your use cases can be assigned to teams or individuals to be
 - Track the progress of the project by use cases delivered.
 - Use cases can be jettisoned or delayed to deliver the highest value soonest.
- Construct tests for your system.
 - Precisely capture a user's requirements and success criteria.

Constructing a Use Case

First Step

Consider the Requirement Given below for a weblog system

REQUIREMENT A.1

The content management system shall allow an administrator to create a new blog account, provided the personal details of the new blogger are verified using the author credentials database.

The First Step is to identify the **Actors**.

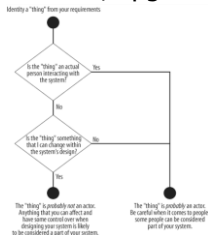
Actors are the things that interact with your system.

Side Note: Terminology in Requirements

- The terms **shall** and **should** have a special and exact meaning.
- A **shall** requirement must be fulfilled.
 - If the feature that implements a shall requirement is not in the final system, then the system does not meet this requirement.
- A **should** requirement implies that the requirement is not critical to the system working but is still desirable.
 - If a system's development is running into problems that will cause delivery delays, then it's often the should requirements that are sacrificed first.

What are Actors

- What are Actors?
 - Actors do not need to be People.
 - It could be external Systems'
 - Actors are like a black box.
 - You cannot change an actor
 - You are not interested in how it works.
- Actors May not only be limited to users within the System.
 - auditors, installers, maintainers, upgraders.



Representing an Actor

An actor is represented using either a “stick man” or a stereotyped box.



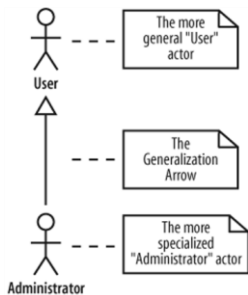
Back to the CMS(Weblog)

- The Administrator interacts with the system to create a new blogger's account.
 - The Administrator interacts with the system and is not part of the system.
 - The Administrator is an actor.
- Authors
- Users

Some actors are related to each other.

Refining Actors

- The Administrator and Author actors are really a special kinds of system users.
- To Represent this relationship and to show that an administrator can do whatever a regular user can do (with some extra additions), a **generalization arrow** can be used.



Second Step – Identifying Use Cases

- Find cases where the system is being used to complete a specific job for an actor.
- can be identified from your user's requirements.
- wordy, blurry definitions in the user requirements document should be distilled into a clear set of jobs for your system.
- *If use cases are truly requirements, then they must have very clear pass/fail criteria*

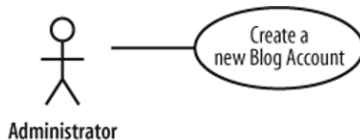
Representing a Use case

- A Use case can be represented using an oval shape with text inside it.
- If we consider the requirements mentioned earlier in A 1



Connecting the Dots

- We can use a communication Line to represent the relationship.



Activity 3

REQUIREMENT A.1

The content management system shall allow an administrator to create a new blog account, provided the personal details of the new blogger are verified using the author credentials database.

Design the Use cases for the above Requirement.

References

- 1 Modeling Requirements: Use Cases — Learning UML 2.0 (oreilly.com)
- 2 Introduction to Requirements Engineering — Requirements Engineering for Software and Systems, 4th Edition (oreilly.com)

Thank You!